CSA4008 - APPLIED MACHINE LEARNING

Presented By

Dr Jasmine Selvakumari Jeya I
Senior Associate Professor
School of Computing Science and Engineering
VIT Bhopal University
Bhopal - Indore Highway, Madhya Pradesh – 466144
jasmineselvakumarijeya@vitbhopal.ac.in

MODULE V

Multi-Layer Perceptron

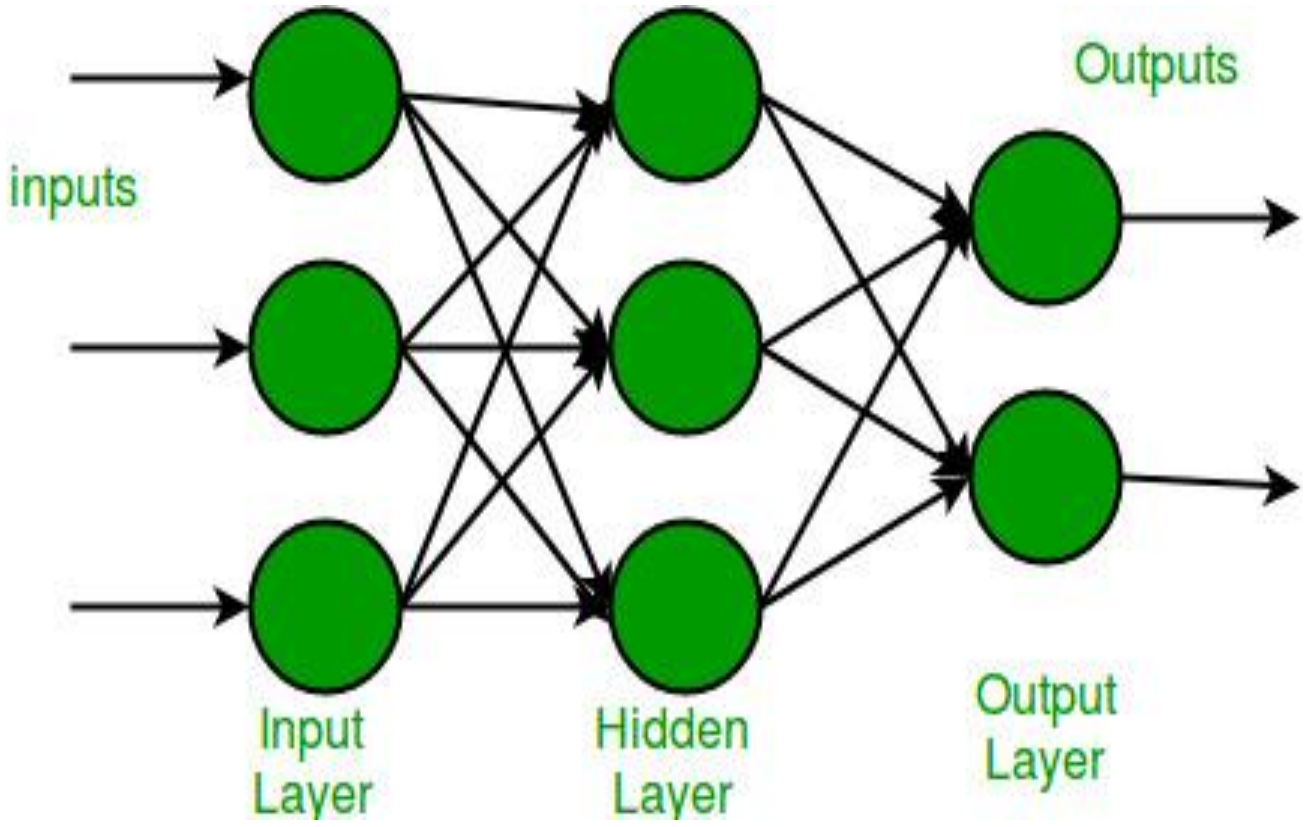
- **Multi-Layer Perceptron (MLP)** consists of fully connected dense layers that transform input data from one dimension to another.
- It is called **multi-layer** because it contains an input layer, one or more hidden layers and an output layer.
- ***The purpose of an MLP is to model complex relationships between inputs and outputs.***

Components of Multi-Layer Perceptron (MLP)

- **Input Layer**: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers**: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer**: The output layer generates the ***final prediction or result.*** If there are multiple outputs, the output layer will have a corresponding number of neurons.

Components of Multi-Layer Perceptron (MLP)

- **Input Layer**: Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers**: MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer**: The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.



- Every connection in the diagram is a representation of the fully connected nature of an MLP.
- ***This means that every node in one layer connects to every node in the next layer.***
- As the data moves through the network each layer transforms it until the final output is generated in the output layer.

Working of Multi-Layer Perceptron

1. Forward Propagation

In **forward propagation** the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

1. Weighted Sum: The neuron computes the weighted sum of the inputs:

$$z = \sum_i w_i x_i + b$$

Where:

- x_i is the input feature.
- w_i is the corresponding weight.
- b is the bias term.

2. Activation Function: The weighted sum z is passed through an activation function to introduce non-linearity. Common activation functions include:

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$
- **ReLU (Rectified Linear Unit)**: $f(z) = \max(0, z)$
- **Tanh (Hyperbolic Tangent)**: $\tanh(z) = \frac{2}{1+e^{-2z}} - 1$

2. Loss Function

Once the network generates an output the next step is to calculate the loss using a loss function. In supervised learning this compares the predicted output to the actual label.

For a classification problem the commonly used **binary cross-entropy** loss function is:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

Where:

- y_i is the actual label.
- $\hat{y}_i$ is the predicted label.
- N is the number of samples.

For regression problems the **mean squared error (MSE)** is often used:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

3. Backpropagation

The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through **backpropagation**:

1. **Gradient Calculation**: The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
2. **Error Propagation**: The error is propagated back through the network, layer by layer.
3. **Gradient Descent**: The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss: $w = w - \eta \cdot \frac{\partial L}{\partial w}$

Where:

- w is the weight.
- η is the learning rate.
- $\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to the weight.

4. Optimization

MLPs rely on optimization algorithms to iteratively refine the weights and biases during training. Popular optimization methods include:

- Stochastic Gradient Descent (SGD): Updates the weights based on a single sample or a small batch of data: $w = w - \eta \cdot \frac{\partial L}{\partial w}$
- Adam Optimizer: An extension of SGD that incorporates momentum and adaptive learning rates for more efficient training:
  - $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \cdot g_t$
  - $v_t = \beta_2 v_{t-1} + (1 - \beta_2) \cdot g_t^2$
- Here g_t represents the gradient at time t and β_1, β_2 are decay rates.

Learning Rules in Neural Network

What are Learning Rules?

- Learning rules are **step-by-step guides** that help a neural network **adjust its weights** to improve accuracy.
- Weights = “decision power” of the network → learning rules decide how they change.

Process:

  - Network makes a prediction.
  - Compares prediction with actual output (error).
  - Adjusts weights based on error.
  - Repeats until error is minimized (iterative process).
- Learning rules are essential for training as they enable the network to adapt and improve over time.

Types of Learning Rules

a) **Hebbian Learning Rule**
b) **Perceptron Learning Rule**
c) **Delta Learning Rule**
d) **Correlation Learning Rule**
e) **Outstar Learning Rule**
f) **Gradient Descent Rule**

(a) Hebbian Learning Rule (Unsupervised)

- "Neurons that fire together, wire together."
- If two neurons activate together → **increase weight** between them.
- If they activate in opposite phases → **decrease weight**.
- No desired output is used → **unsupervised learning**.

$$\Delta w_{ij} = \eta x_i y_j$$

- Where x_i, y_j = input and output activations.
- Suitable for discovering relationships between inputs.

(b) Perceptron Learning Rule (Supervised)

- Adjust weights only when prediction is wrong.
- Weights initialized randomly, then updated to minimize error.

$$w_{new} = w_{old} + \eta(d - y)x$$

- where d=desired output, y=actual output.
- Used for simple binary classification problems.
- η=Learning rate (controls step size of weight update)
- *X*=Input from neuron

(c) Delta Learning Rule / Widrow-Hoff Rule (Supervised)

- Adjust weight proportionally to **error × input**.
- Minimize **mean squared error (MSE)** between predicted and desired output.

$$\Delta w_{ij} = \eta x_i d_j$$

Δw_{ij} → Change in weight from input neuron i to output neuron j.

η → Learning rate (controls how big the weight adjustment step is).

x_i → Input signal from neuron i.

d_j → Desired output (target value) of neuron j.

- Works well for **linear activation neurons** without hidden layers.
- Basis for **gradient descent** and backpropagation.

(d) Correlation Learning Rule (Supervised)

- Similar to Hebbian rule but uses **desired output (d)** instead of actual output.
- Strengthens connection if desired output and input are correlated.

$$\Delta w_{ij} = \eta x_i d_j$$

- Starts with weights = 0 and updates using labeled data.

(e) Outstar Learning Rule (Supervised)

- Used when neurons are arranged in **layers**.
- Adjusts weights so that outputs match **desired response** for that layer.
- Suitable for Kohonen-type networks.
- Ensures that all neurons in a layer produce the correct target output.

f) Gradient Descent Rule

- Most common rule in deep learning.
- Uses gradient of loss function w.r.t. weights:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

- where L = loss function.
- This is the basis for **backpropagation**.

- **Hebbian →** Unsupervised, learns correlations.
- **Perceptron & Delta →** Supervised, minimize error.
- **Correlation Rule →** Supervised, uses desired outputs.
- **Outstar Rule →** Supervised, layer-level weight update.
- All rules iteratively improve network performance.

Backpropagation in Neural Network

- Backpropagation (Backward Propagation of Errors) is a method used to train a neural network.
- Reduce the difference between predicted output and actual output by adjusting **weights** and **biases**.

How it Works:

- Adjusts weights and biases iteratively to minimize cost function.
- In each epoch, parameters are updated by reducing loss, following the error gradient.
- Uses optimization algorithms like **Gradient Descent** or **Stochastic Gradient Descent**.
- Computes gradients using **chain rule** from calculus to handle complex layers.

- **Efficient Weight Update:** Computes gradient of loss function w.r.t. each weight using chain rule → efficient weight updates.
- **Scalability:** Works for multi-layer, complex networks → makes deep learning feasible.
- **Automated Learning:** Model automatically adjusts weights to improve performance.

Working of Backpropagation Algorithm

1. Forward Pass

- Input data is fed into the input layer.
- Inputs + weights → passed to hidden layers.
- Bias is added before activation function is applied.
- Hidden layer computes weighted sum (`a`) → applies activation function (e.g., ReLU) → output (`o`).
- Output layer applies softmax to convert outputs into probabilities for classification.

2. Backward Pass

- Error = difference between predicted and actual output.
- Error is propagated back to adjust weights & biases.
- **Mean Squared Error (MSE):**

$$MSE = (Predicted\ Output - Actual\ Output)^2$$

- Gradients computed using chain rule indicate how much to adjust each weight/bias.
- Backward pass continues layer by layer → network learns and improves performance.
- Activation function derivative is critical in gradient calculation.

Example of Backpropagation

Forward Propagation

1. **Initial Calculation:**

$$a_j = \sum (w_{i,j} \times x_i)$$

Where:

- a_j: Weighted sum at node
- $w_{i,j}$: Weight from input i to neuron j
- x_i: Input value

Neuron output:

$$o_j = \text{activation function}(a_j)$$

2. **Sigmoid Function:**

$$y_j = \frac{1}{1 + e^{-a_j}}$$

Consider a neural network with two inputs $x_1 = 0.35$ and $x_2 = 0.7$, connected to two hidden layer neurons h_1, h_2 with the following weights:

$$w_{1,1} = 0.2, \quad w_{2,1} = 0.2, \quad w_{1,2} = 0.3, \quad w_{2,2} = 0.3$$

The hidden layer outputs y_3, y_4 are connected to the output neuron O_3 with weights:

$$w_{1,3} = 0.3, \quad w_{2,3} = 0.9$$

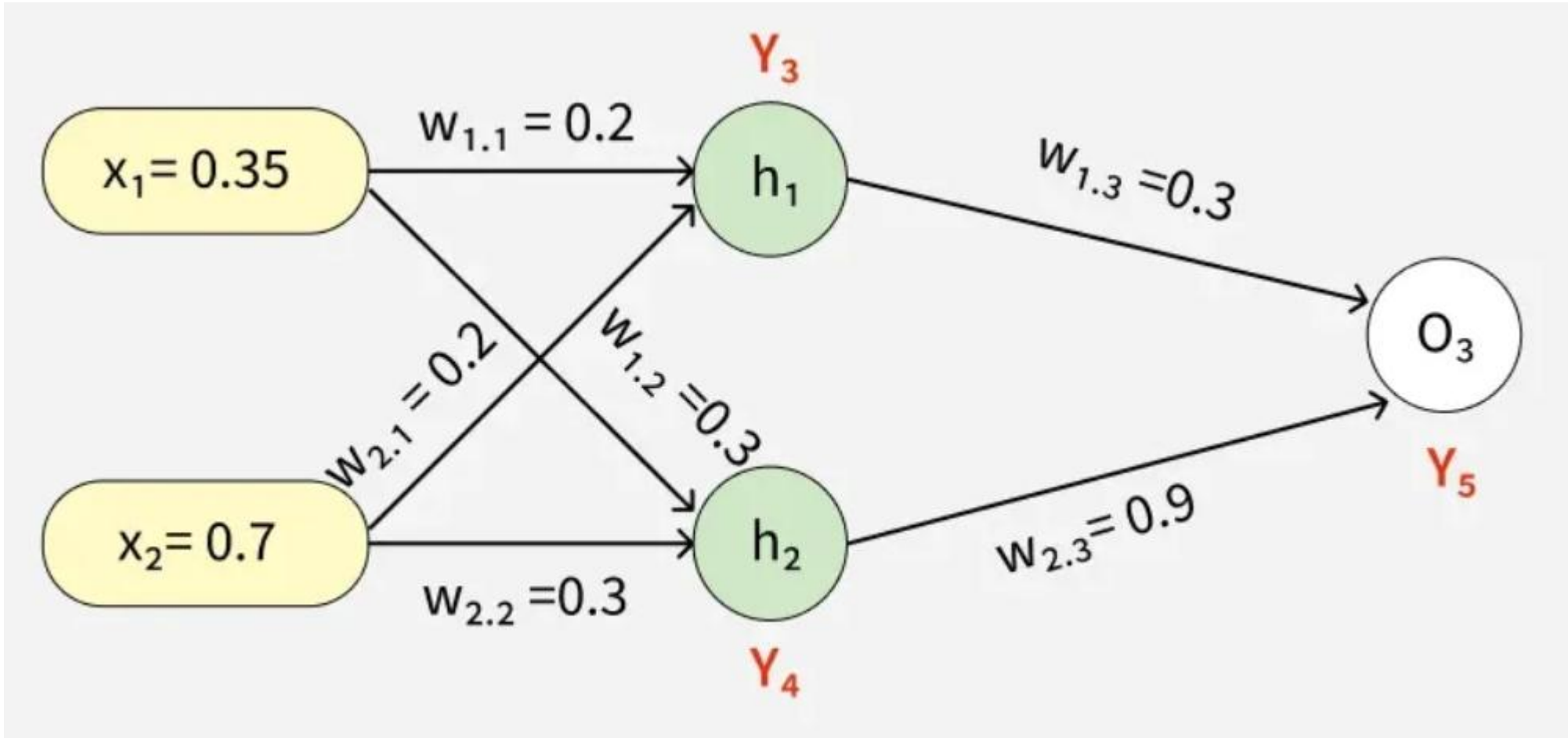
Use the **sigmoid activation function**:

$$y = \frac{1}{1 + e^{-a}}$$

where a is the weighted sum.

1. Compute the weighted sum and outputs y_3, y_4, y_5.
2. If the target output is $y_{target} = 0.5$, calculate the **error** using:

$$Error = y_{target} - y_5$$


Y3
x1= 0.35
w1.1 = 0.2
h1
w1.3 =0.3
O3
w2.1 =0.2
w1.2 =0.3
Y5
x2= 0.7
h2
w2.3= 0.9
w2.2 =0.3
Y4

$$x_1 = 0.35,\ x_2 = 0.7$$

$$w_{1,1} = 0.2,\ w_{2,1} = 0.2,\ w_{1,2} = 0.3,\ w_{2,2} = 0.3$$

$$w_{1,3} = 0.3,\ w_{2,3} = 0.9$$

Sigmoid: $\sigma(a) = \dfrac{1}{1 + e^{-a}}$

Target: $y_{\text{target}} = 0.5$

At h1 node

$$
\begin{aligned}
a_1 &= (w_{1,1}x_1) + (w_{2,1}x_2) \\
&= (0.2 * 0.35) + (0.2 * 0.7) \\
&= 0.21
\end{aligned}
$$

Once we calculated the a_1 value, we can now proceed to find the y_3 value:

$$y_j = F(a_j) = \frac{1}{1+e^{-a_1}}$$

$$y_3 = F(0.21) = \frac{1}{1+e^{-0.21}}$$

$$y_3 = 0.56$$

Similarly find the values of y_4 at $\mathbf{h_2}$ and y_5 at O_3

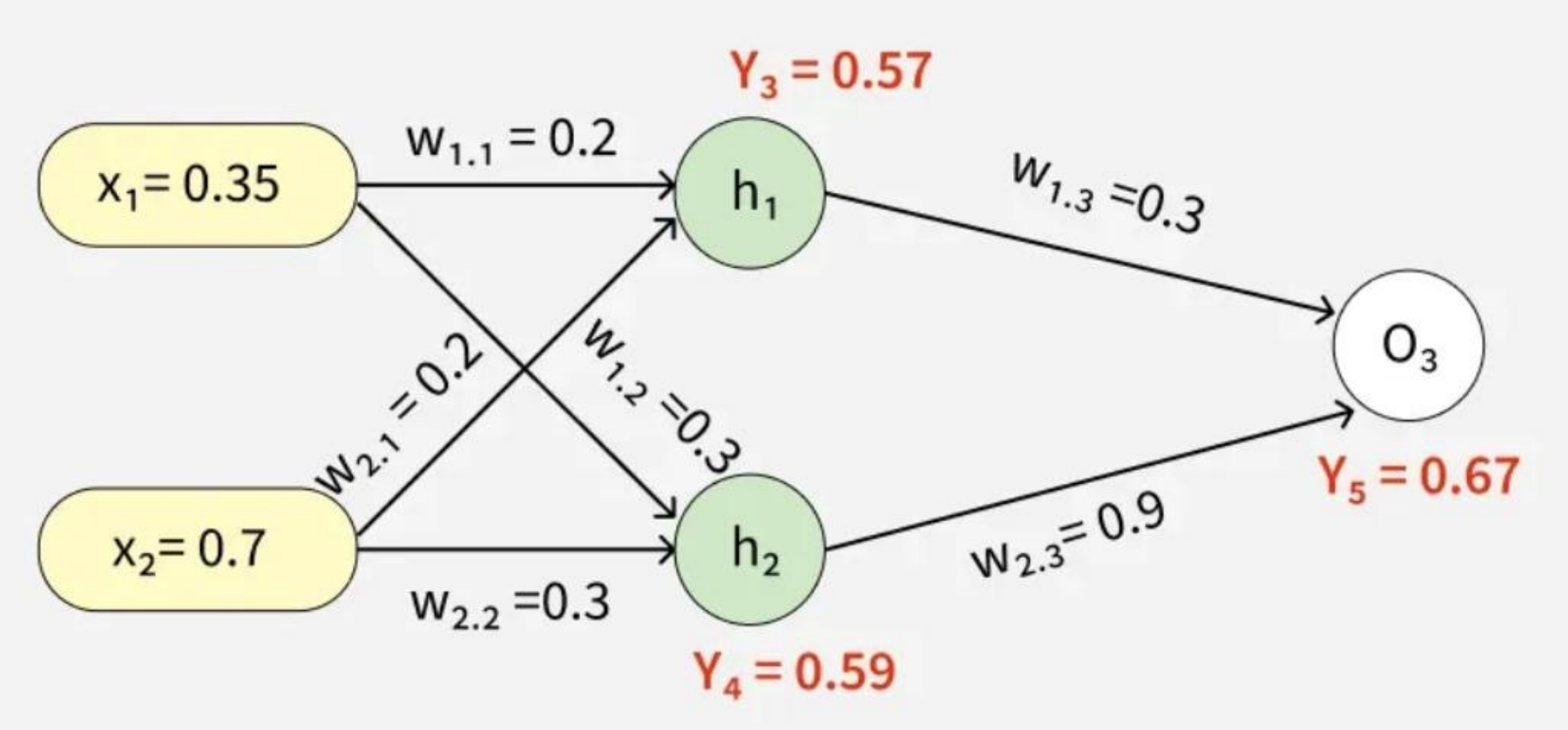
$$a_2 = (w_{1,2} * x_1) + (w_{2,2} * x_2) = (0.3 * 0.35) + (0.3 * 0.7) = 0.315$$

$$y_4 = F(0.315) = \frac{1}{1+e^{-0.315}}$$

$$a3 = (w_{1,3} * y_3) + (w_{2,3} * y_4) = (0.3 * 0.57) + (0.9 * 0.59) = 0.702$$

$$y_5 = F(0.702) = \frac{1}{1+e^{-0.702}} = 0.67$$

Answer:


$Y_3 = 0.57$
$x_1 = 0.35$
$w_{1.1} = 0.2$
h_1
$w_{1.3} = 0.3$
O_3
$w_{2.1} = 0.2$
$w_{1.2} = 0.3$
$Y_5 = 0.67$
$x_2 = 0.7$
h_2
$w_{2.3} = 0.9$
$w_{2.2} = 0.3$
$Y_4 = 0.59$

4. Error Calculation

Our actual output is 0.5 but we obtained 0.67. To calculate the error we can use the below formula:

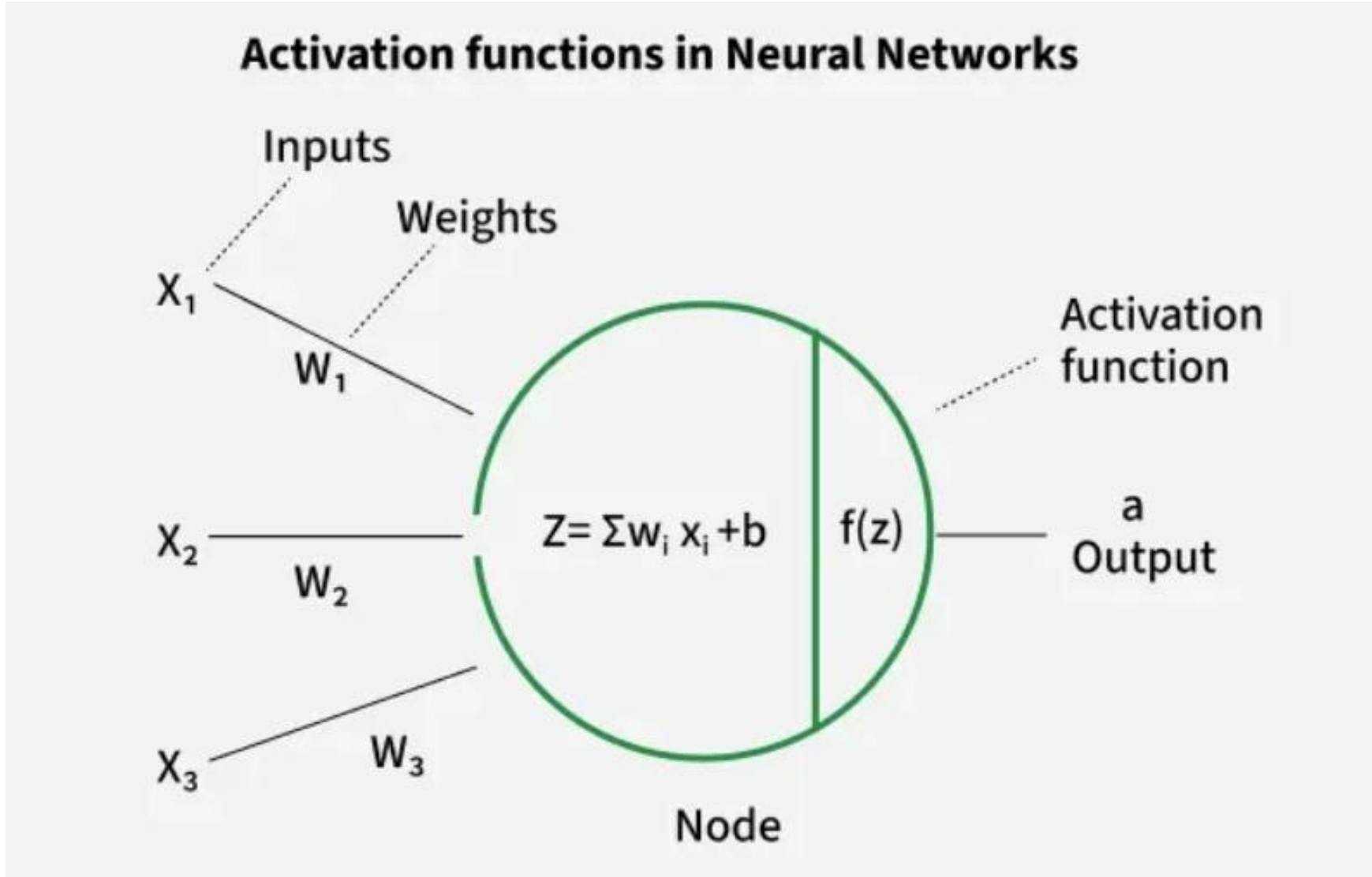
$$Error_j = y_{target} - y_5$$

$$\Rightarrow 0.5 - 0.67 = -0.17$$

Using this error value we will be backpropagating.

Activation functions in Neural Networks

- Choosing the activation function for hidden and output layers is crucial as it directly affects the network's ability to model complex patterns.
- An activation function is applied to a neuron's output, transforming it and introducing non-linearity into the model.
- **Importance of Non-Linearity:** Without non-linear activation functions, a neural network behaves like a simple linear regression, regardless of the number of layers.
- **Neuron Activation:** Activation functions determine whether a neuron should fire by computing the weighted sum of inputs plus a bias, enabling complex decisions and predictions.


Activation functions in Neural Networks
Inputs
Weights
X_1
W_1
X_2
W_2
X_3
W_3
$Z= \Sigma w_i x_i + b$
f(z)
Activation function
a
Output
Node

This diagram represents **a single neuron (node) in a neural network** and how the **activation function** works.

Inputs (X1,X2,X3):

These are the signals or data fed into the neuron from either the input layer or the previous layer of the network.

Weights (W1,W2,W3):

Each input has a weight associated with it. The weight determines the importance or contribution of that input to the neuron's output.

Weighted Sum and Bias ($Z=\sum WiXi+b$):

The neuron computes a weighted sum of all its inputs and adds a bias term (b) to shift the activation function if needed. This is the linear combination part.

Activation Function (f(z)):

The weighted sum is passed through a ***non-linear activation function***. This introduces non-linearity, allowing the neural network to learn complex patterns. Examples of activation functions include ***ReLU, Sigmoid, and Tanh.***

Output (a):

The result of the activation function is the output of the neuron. This output is then sent to the next layer in the network or used as the final prediction.

Introducing Non-Linearity in Neural Networks

- Non-linearity means the relationship between input and output is not a straight line; output does not change proportionally with input.
- **Example Function**: A common non-linear function is ReLU, defined as:

$$\sigma(x)=\max(0,x)$$

- **Example**: Suppose we want to classify apples and bananas based on shape and color.
  - Using a **linear function** can only separate them with a straight line.
  - Real-world data is often more complex (e.g., overlapping colors, varying lighting).
- **Benefit of Non-Linearity**: By using non-linear activation functions like **ReLU, Sigmoid, or Tanh**, the network can form **curved decision boundaries**, enabling it to separate complex patterns correctly.

Effect of Non-Linearity

- The inclusion of the ReLU activation function σ allows h1 to introduce a non-linear decision boundary in the input space. This non-linearity enables the network to learn more complex patterns that are not possible with a purely linear model such as:
  - Modeling functions that are not linearly separable.
  - Increasing the capacity of the network to form multiple decision boundaries based on the combination of weights and biases.

Example

- We want to build a simple neural network to classify apples vs bananas.

Fruit	Weight (x1)	Color (x2)
Apple	150	7
Apple	160	6
Banana	120	4
Banana	130	5

- We want to build a simple neural network to classify apples vs bananas.

1. Linear Activation (No Non-Linearity)

Suppose the neuron just sums the weighted inputs:

$$y = w_1 x_1 + w_2 x_2 + b$$

Take arbitrary weights: $w_1 = 0.05, w_2 = 0.5, b = -12$

For **Apple (150, 7)**:

$$y = 0.05 * 150 + 0.5 * 7 - 12 = 7.5 + 3.5 - 12 = -1$$

For **Banana (120, 4)**:

$$y = 0.05 * 120 + 0.5 * 4 - 12 = 6 + 2 - 12 = -4$$

- Both outputs are **negative**, so if we use a threshold of 0 for classification, the neuron **cannot separate apples and bananas properly**.
- Linear neurons can only draw **straight-line boundaries** in this 2D space.

2. Adding Non-Linearity (ReLU Activation)

ReLU function:

$$\sigma(x) = \max(0, x)$$

Apply ReLU to outputs:

- Apple: $y = \max(0, -1) = 0$
- Banana: $y = \max(0, -4) = 0$

Now, let's add a **hidden layer** with **two neurons**:

- **Neuron 1**: $y_1 = w_{11}x_1 + w_{12}x_2 + b_1 = 0.1x_1 - 10$
- **Neuron 2**: $y_2 = w_{21}x_1 + w_{22}x_2 + b_2 = 0.1x_2 - 1$

Fruit	y1	y2
Apple	150*0.1-10=5 → ReLU(5)=5	7*0.1-1=-0.3 → ReLU(0)=0
Banana	120*0.1-10=2 → ReLU(2)=2	4*0.1-1=-0.6 → ReLU(0)=0

Now the outputs of these two neurons are **different combinations**. A second layer can now combine y1,y2 to draw a **curved boundary**, allowing correct separation of apples and bananas.

Why is Non-Linearity Important in Neural Networks?

- Neural networks use **neurons with weights, biases, and activation functions** to process data.
- During learning, **weights and biases are updated** using backpropagation, which depends on gradients.
- **Activation functions provide these gradients**, enabling the network to learn effectively.
- Without non-linearity, networks can only solve **simple, linearly separable problems**.
- Non-linear activation functions allow networks to **model complex data** and learn **abstract patterns**.
- They add **flexibility**, making deep learning capable of handling **advanced tasks**.

Example:

Suppose we have a tiny neural network with **one input** x, one weight w, no bias, and a simple linear function:

$$y = w \cdot x$$

Let's say the actual output we want is $y_{\text{true}} = 10$, the input is $x = 2$, and the current weight is $w = 3$.

1. **Calculate predicted output:**

$$y = w \cdot x = 3 \cdot 2 = 6$$

2. **Define error (loss):**

$$\text{Loss} = (y_{\text{true}} - y)^2 = (10 - 6)^2 = 16$$

3. **Find the gradient:**

   The gradient is the derivative of Loss with respect to the weight w:

$$\frac{d\text{Loss}}{dw} = 2 \cdot (y - y_{\text{true}}) \cdot x$$

Plug in the numbers:

$$\frac{d\text{Loss}}{dw} = 2 \cdot (6 - 10) \cdot 2 = 2 \cdot (-4) \cdot 2 = -16$$

4. **Update the weight using gradient descent:**

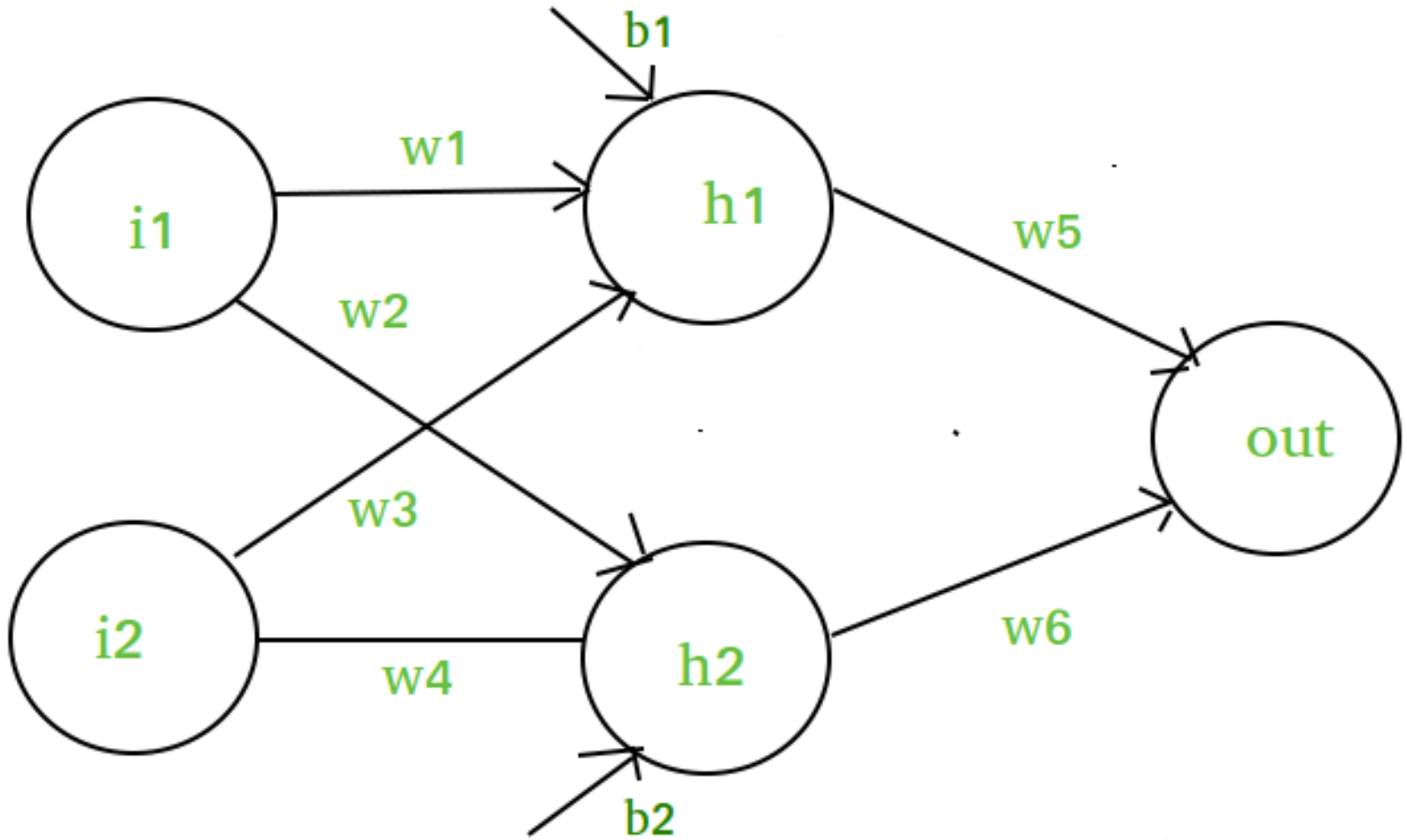
$$w_{\text{new}} = w - \text{learning rate} \cdot \text{gradient}$$

Assume learning rate = 0.1:

$$w_{\text{new}} = 3 - 0.1 \cdot (-16) = 3 + 1.6 = 4.6$$

Mathematical Proof of Need of Non-Linearity in Neural Networks

1. **Input Layer:** Two input nodes: i_1 and i_2.
2. **Hidden Layer:** Two neurons: h_1 and h_2.
3. **Output Layer:** One output neuron: out.
4. **Weights:**
   - w_1, w_2 connect inputs to hidden neurons.
   - w_5 connects hidden neurons to the output neuron.
5. **Biases:**
   - b_1 for hidden neurons.
   - b_2 for the output neuron.
6. **Purpose:** This setup shows how inputs are combined and transformed through the network.
7. **Non-Linearity Role:** Adding a non-linear activation function to hidden neurons allows the network to learn **complex patterns** that linear combinations alone cannot capture.


b1
w1
i1
h1
w5
w2
out
w3
i2
h2
w6
w4
b2

The input to the hidden neuron h_1 is calculated as a weighted sum of the inputs plus a bias:

$$h_1 = i_1.w_1 + i_2.w_3 + b_1$$

$$h_2 = i_1.w_2 + i_2.w_4 + b_2$$

The output neuron is then a weighted sum of the hidden neuron's output plus a bias:

$$\text{output} = h_1.w_5 + h_2.w_6 + \text{bias}$$

In order to add non-linearity, we will be using sigmoid activation function in the output layer:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\text{final output} = \sigma(h_1.w_5 + h_2.w_6 + \text{bias})$$

$$\text{final output} = \frac{1}{1+e^{-(h_1.w_5+h_2.w_6+\text{bias})}}$$

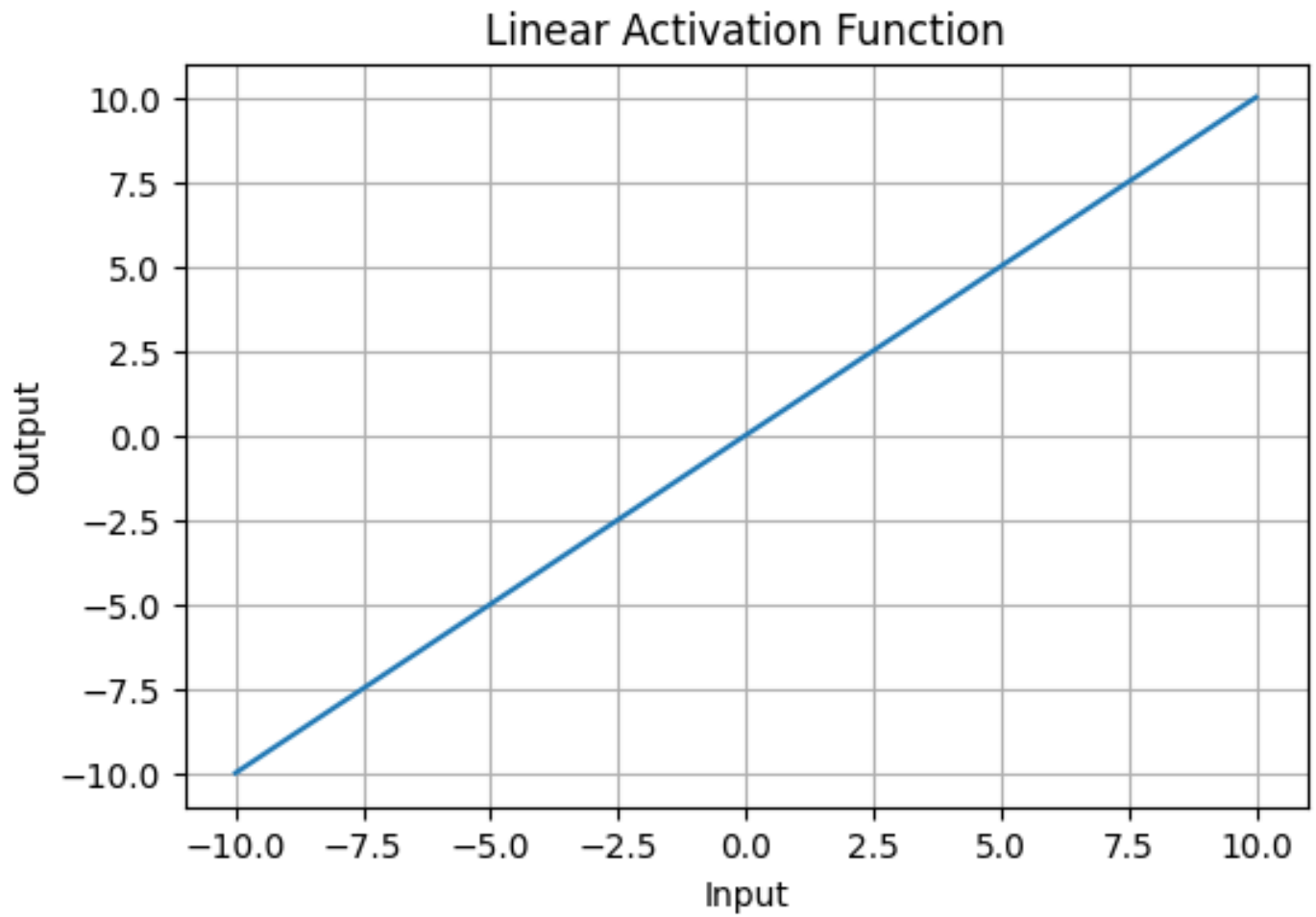
This gives the final output of the network after applying the sigmoid activation function in output layers, introducing the desired non-linearity.

Types of Activation Functions in Deep Learning

1. **Linear Activation Function**
2. **Non-Linear Activation Functions**
   - **Sigmoid Function**
   - **Tanh Activation Function**
   - **ReLU** (Rectified Linear Unit) **Function**
3. **Exponential Linear Units**
   - **Softmax Function**
   - **SoftPlus Function**

1. Linear Activation Function

- Linear activation function is defined as **y=x**, resembling a straight line.
- No matter how many layers the network has, if all use linear activation, the output is just a **linear combination of inputs**.
- The **output range** spans from $-\infty$ to $+\infty$.
- Linear activation is typically used **only in the output layer**.
- Using linear activation across all layers **limits the network's ability to learn complex patterns**.
- Linear functions are useful for specific tasks but should be **combined with non-linear activations** to enhance learning and predictive power.

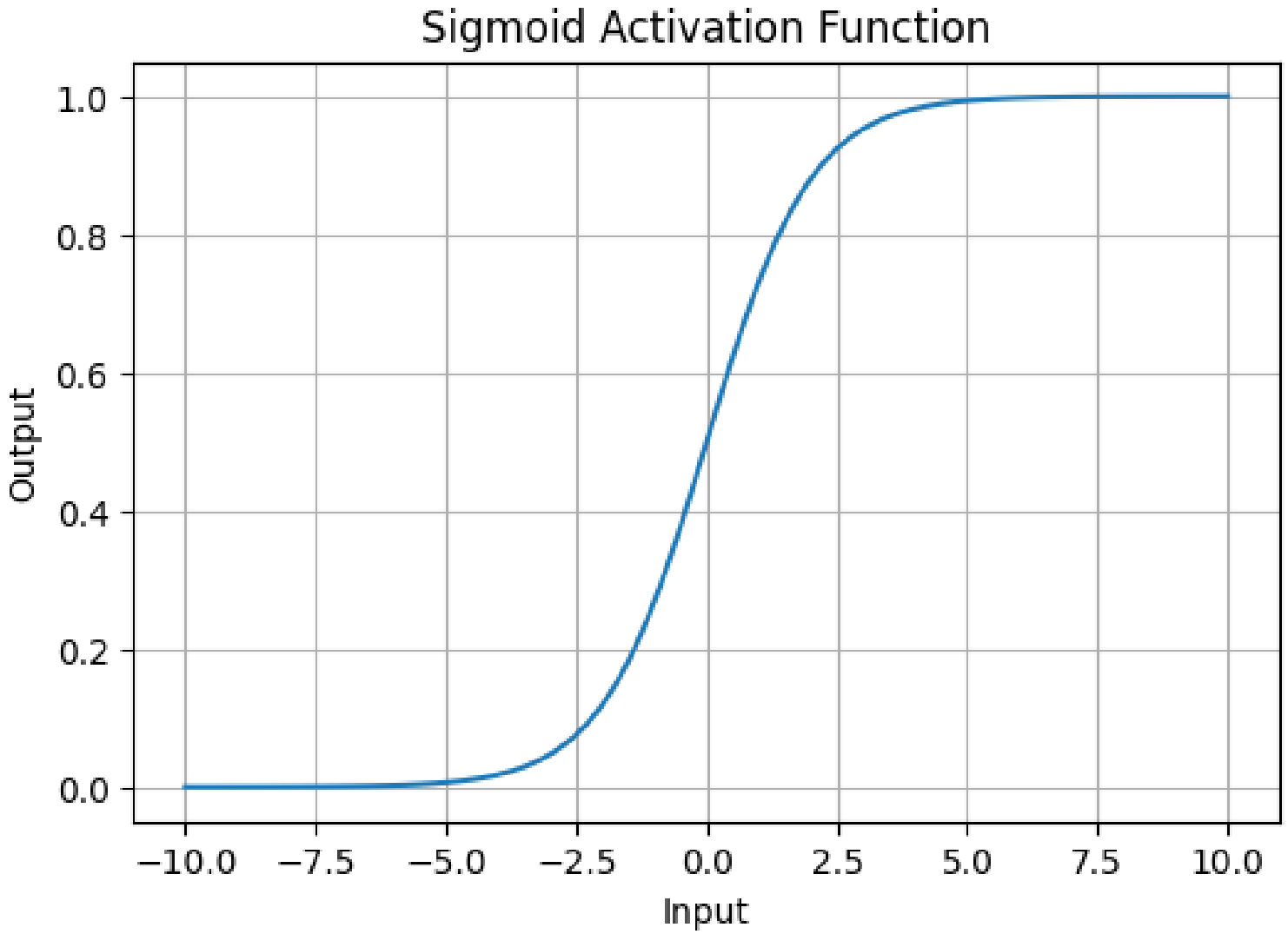

Linear Activation Function
Output
Input
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

2. Non-Linear Activation Functions

1. Sigmoid Function

Sigmoid Activation Function is characterized by 'S' shape. It is mathematically defined as $A = \frac{1}{1+e^{-x}}$. This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.

- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence useful for binary classification.
- The function exhibits a steep gradient when x values are between -2 and 2. This sensitivity means that small changes in input x can cause significant changes in output y which is critical during the training process.


Sigmoid Activation Function
1.0
0.8
0.6
0.4
0.2
0.0
Output
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
Input

2. Tanh Activation Function

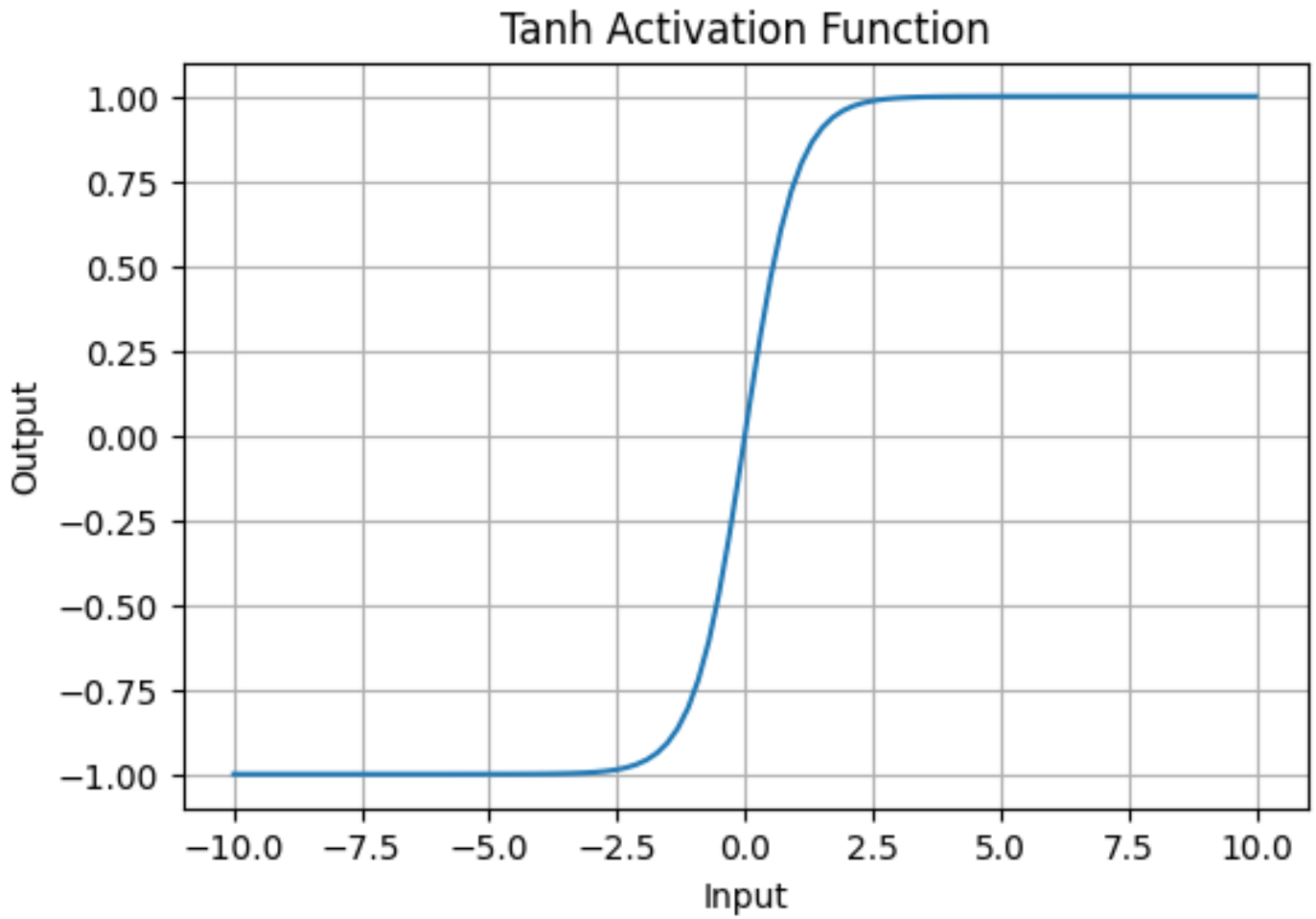
Tanh function (hyperbolic tangent function) is a shifted version of the sigmoid, allowing it to stretch across the y-axis. It is defined as:

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1.$$

Alternatively, it can be expressed using the sigmoid function:

$$\tanh(x) = 2 \times \text{sigmoid}(2x) - 1$$

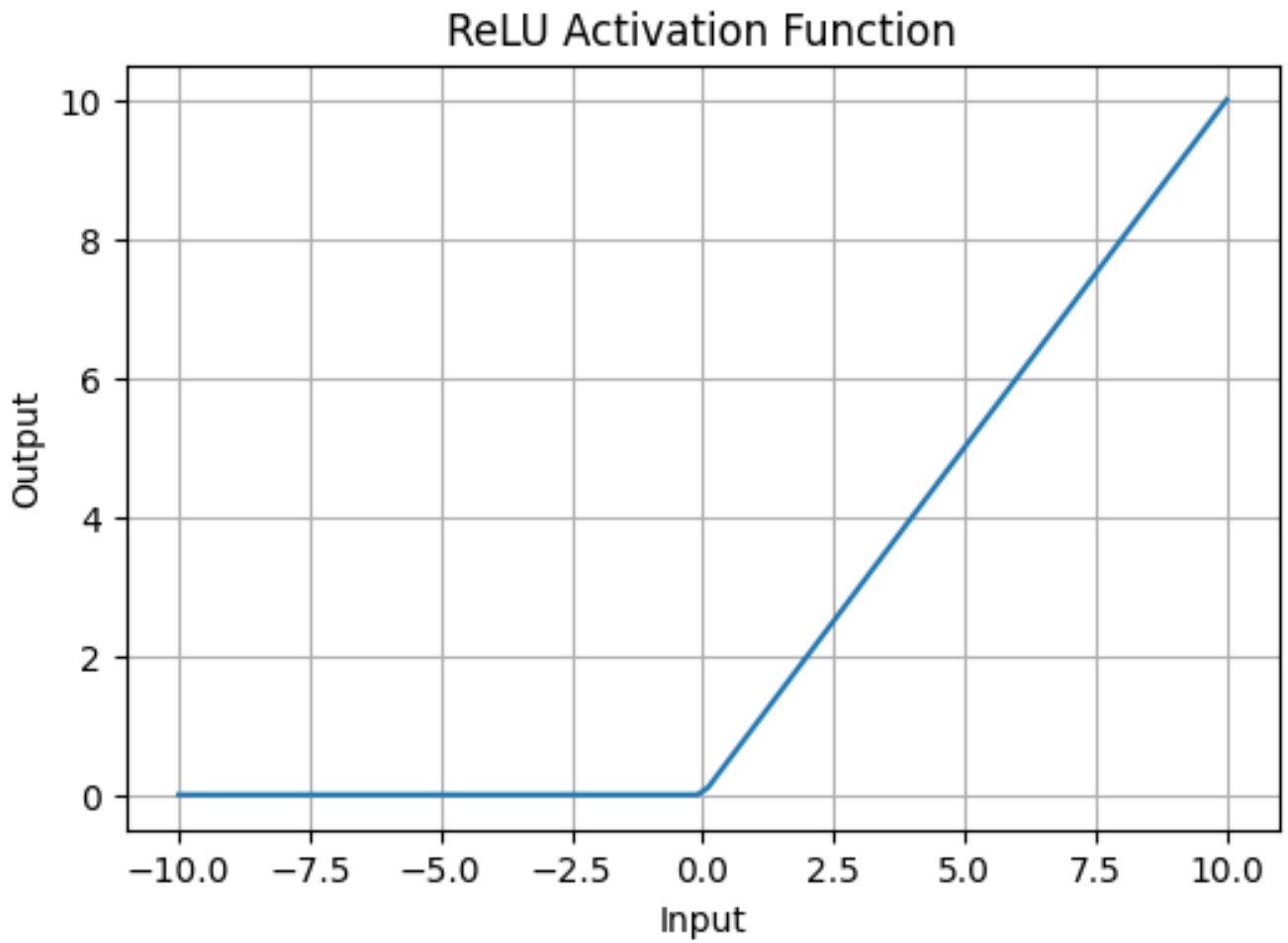
- **Value Range**: Outputs values from -1 to +1.
- **Non-linear**: Enables modeling of complex data patterns.
- **Use in Hidden Layers**: Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.


Tanh Activation Function
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Output
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
Input

3. **ReLU** (Rectified Linear Unit) **Function**

ReLU activation is defined by $A(x) = \max(0, x)$, this means that if the input x is positive, ReLU returns x, if the input is negative, it returns 0.

- **Value Range**: $[0, \infty)$, meaning the function only outputs non-negative values.
- **Nature**: It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.
- **Advantage over other Activation:** ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.


ReLU Activation Function
Output
Input
10
8
6
4
2
0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

THANK YOU!!!